PROJECT-BASED LEARNING

PHASE-I EVALUATION

ExamLens -:Syllabus and PYQS Analyzer

Team ID: T309

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: T309
Semester: 3rd
Section: D,F,ML1,H
Domain: CSE

Team Members

1. *Madhav Nauriyal– 2510011324*
2. *Aarav Sharma – 2510012730*
3. *Ishant – 2510370272*
4. *Sasmit Sinha – 2510012002*

Mentor

Ms. Shrishti

Interactions completed: _____

Problem Statement & Motivation

Problem Statement

Students preparing for university entrance exams lack a structured, data-driven way to identify high-priority topics. The current manual approach is inefficient and failing students on a large scale.

Motivation

- *It took entire nights to manually analyze just a few past papers.*
- *Failing to prioritize high-yield topics hurts exam performance.*
- *Manual polling is inefficient, error-prone, and highly repetitive.*

Target Users / Stakeholders

- *Primary users: Undergraduate students*
- *Secondary users: Faculty and mentors*
- *Expected benefit: Prioritize data over intuition for your final preparations.*

Objectives

Project Objectives

1. *ExamLens: An offline CLI generating data-driven study priorities from past exams.*
2. *Develop the C core for matching, deduplication, and top-k ranking.*
3. *ntegrate Data Structures with C++ OOP*
4. *Verify every output against a manually counted*
5. *Conduct a live demonstration of the tool and present the scope for Phases II and III, which include PDF extraction and NLP-based matching*

Key Objective: : *Develop a CLI tool that perfectly matches manual topic counts. .*

Proposed Solution

Proposed Approach

1. **Input/Data:** *Plain-text syllabus + 5–10 years of PYQs.*
2. **Processing:** *Object Parsing → Trie/KMP Matching → Min-Heap Ranking.*
3. **System Logic: C Core** *(Trie, KMP, Heap)* → **C++ Layer** *(OOP parsing & analysis).*
4. **Storage:** *100% in-memory (Hash Maps & Trie) — no external DB.*
5. **Output:** *Tiered priority reports (Very High to Low).*

Key Features

- *Automated topic-to-question mapping*
- *Near-duplicate question detection*
- *Unit-wise and topic-wise trend classification*
- *Fully offline CLI — runs on a lab machine*
- *Composite importance scoring*

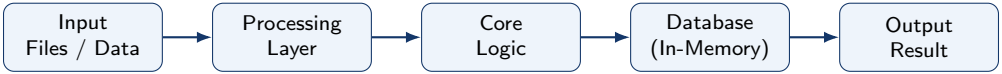
It substitutes manual counting with an objective, repeatable, evidence-based assessment of the specific curriculum and examinations a student has.

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C (TCS 302)	Hash maps, Trie, Min-heap, Arrays, sorting	C core (`examlens_core.h`)
OOPs with C++ (TCS 307)	KMP / string matching, Classes & objects, Encapsulation, Abstraction, Polymorphism, Inheritance, Virtual functions	Parser & Analyzer class hierarchy, ReportWriter, STL, File handling

Note: Demonstrating meaningful integration of both core subjects in the ExamLens project.

System Architecture / Workflow



Major Components

- **Parser:** SyllabusParser & QuestionParser (C++) extract topics, years, and marks from plain-text/PDFs.
- **Topic Matcher:** Trie + KMP matching (C core) maps questions to syllabus units.
- **Analyzer & ReportWriter:** Subclasses combine frequency, recency, & marks into a 0–100 priority score via min-heap.

Data / Control Flow

- **Data Pipeline:** Input → Parser → Topic Matcher → Analyzer → Final Report.
- **Module Interaction:** C++ classes orchestrate logic, calling the C core (`examlens_core.h`) which manages state without an external DBMS.
- **Expected Output:** `analyze` command outputs a ranked `report.md` and CSV totals.

Technology Stack & Methodology

Technology Stack

- **Programming:** C (Core Logic) & C++ (OOP Layer)
- **Database:** 100% In-Memory (Trie/Hash) — No DBMS
- **Tools:** CLI, Python (for PDF ingestion)
- **IDE/Compiler:** VS Code, GCC/G++
- **Version Control:** Git & GitHub

Development Methodology

1. **Requirement Analysis:** Syllabus & PYQ scoping
2. **System Design:** Architecture & Data Structures
3. **Implementation:** C Core followed by C++ Layer
4. **Testing:** CLI outputs vs. manual counting
5. **Evaluation:** Speed and accuracy assessment

Stack chosen for maximum performance and zero external dependencies, allowing offline execution

Initial Progress & Team Contribution

Progress Achieved

- **Problem Validation:** Analyzed pain points of manual PYQ mapping.
- **Architecture:** Finalized C core and C++ OOP integration strategy.
- **Data Structures:** Designed custom Trie, Min-Heap, and Hash Maps.
- **Data Collection:** Gathered test dataset (e.g., TMA 316 PYQs).
- **Prototyping:** Initial CLI parsing and text extraction implemented.

Individual Contribution

Member	Role	Contribution
Madhav	Core Logic	Researching & coding C structures (Trie, Heap)
Aarav	OOP Design	Learning & applying C++ classes for parsing
Ishant	Data & QA	Collecting PYQs, formatting datasets, and testing
Sasmit	I/O & Docs	Handling file read/write, output reports, and docs

Roadmap, Expected Outcomes & References

Roadmap

1. **Phase-I:** Proposal, Architecture & Setup *(Current)*
2. **Phase-II:** Core C Logic & C++ OOP Development
3. **Phase-III:** CLI Integration, Testing & Final Release

Expected Outcomes

- **Functional CLI:** Fully offline prototype running on lab machines.
- **Actionable Reports:** Auto-generated, priority-ranked syllabus analysis.
- **Time Efficiency:** Replacing overnight manual counting with instant results.
- **Scalability:** Easily adaptable to any university subject's PYQs.

References

1. Horowitz & Sahni, *Fundamentals of Data Structures*, Galgotia
2. Tenenbaum, Langsam & Augenstein, *Data Structures Using C and C++*, PHI
3. Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, 3rd ed., MIT Press
4. Knuth, Morris & Pratt, "Fast pattern matching in strings," *SIAM J. Computing*, 6(2), 1977
5. Gamma, Helm, Johnson & Vlissides, *Design Patterns*, Addison-Wesley, 1994 (Strategy pattern)
6. GEU syllabus documents for TCS 302, TCS 307, TMA 316; TMA 316 PYQs (Oct 2022 – Dec 2023)

Thank You!